Chapter 13: Applying the ADM

Study Guide, based on Mastering the TOGAF Standard by Eric Jager (2025), Chapter 13 (pp. 561-587)

Overview

Chapter 13 (pp. 561-587) shows how the ADM is adapted to real situations, following the Standard's "Applying the ADM" document. Section 13.1 introduces three scenarios (p. 562). Section 13.2 explains using TOGAF with different **architecture styles** (SOA, microservices, zero-trust) (pp. 563-564). Section 13.3, the core of the chapter, explains **iteration**: the three ways to iterate, the four iteration groups, and baseline-first vs. target-first cycles (pp. 564-580). Section 13.4 covers **architecture partitioning**: dividing the architecture among teams (pp. 580-583). Section 13.5 summarizes (pp. 583-587). Leave with this idea: the ADM graphic is a picture of *information flows*, not an activity sequence. "The Architecture Development Method should not be understood as a process model" (pp. 565-566). Chapter 13 is part of the GREEN and BLUE reading tours (p. 561).

> **Reading note:** The three ways of iterating, the four iteration groups, Table 13-1, and partitioning follow the Standard. The Initiative A/B budget scenario (pp. 569-571) is Jager's example. Keep two lists apart: three *ways* to iterate (whole cycle, within a cycle, Architecture Capability) and four iteration *groups* (Architecture Capability, Architecture Development, Transition Planning, Architecture Governance).

Detailed Section Breakdown

Chapter Opener (p. 561)

- The ADM is step-by-step but "not to be confused with a waterfall approach". It addresses each sub-area in a focused way, works with many architecture styles, and its iterative nature lets it respond to different situations (p. 561).

13.1 Various Architecture Development Method Approaches (p. 562)

- **Tailorability** suits the ADM to different process styles and specialized architectures such as Security Architecture, for "almost any situation" (p. 562).
- **Three scenarios covered:** different architecture styles, iteration, and partitioning. They cover most applications (p. 562).

13.2 Using the TOGAF Framework with Different Architecture Styles (pp. 563-564)

- **Versatile across styles.** E.g., SOA, Microservices, Zero-Trust, which differ in focus, form, techniques, and history. Several styles can coexist in one landscape (p. 563).
- **Approach.** (1) Identify the style's unique characteristics. (2) Decide how to address them, usually by adapting models, views, and tools, not by changing the framework (p. 563).
- **Where it shows up.** In B, C, and D, choose resources and viewpoints for the style. Styles may add elements, emphasize others, change notation, or focus on certain stakeholders, which often means extending the Enterprise Metamodel and Content Framework (p. 563).
- **Preliminary revisited.** A predominant style may require revisiting the Preliminary Phase to adjust the Architecture Capability. Style-specific reference and maturity models help (p. 564).

13.3 Applying Iteration to the Architecture Development Method (pp. 564-580)

- **Iterative by nature.** Phases can be revisited as requirements, feedback, and context change (p. 564).
- **The graphic is misread.** Figure 13-1 is "frequently misinterpreted as a linear, waterfall-style process". It is "a logical method that groups key activities to clarify their relationships and information flow" (p. 565).
- **Any activity is a phase.** E.g., working on a roadmap *is* doing Phase E, with its mandatory inputs and outputs (p. 566).

13.3.1 Different Ways to Apply Iteration (pp. 566-572)

Way to iterate	What it means	Used for	Page
Across the whole ADM cycle	Run the entire cycle repeatedly for different levels or scopes (enterprise vs. project), or layer by layer	Building a comprehensive Architecture Landscape. Moving from high-level EA to detailed architectures. Agile or incremental	pp. 566-567
Within an ADM cycle	Loop back and forth between two or more phases as insights emerge	Managing B/C/D interdependencies. Converging on a detailed target without higher-level context	pp. 567-569
To manage the Architecture Capability	Re-run the Preliminary Phase and Phase A	Adjusting the capability, e.g., after a Phase H change request	pp. 571-572

- **Common within-cycle groups (Figure 13-2).** Architecture Development (B, C, D baseline and target), Transition Planning (to reach an executable roadmap or migration plan when scope, strategy, or requirements shift), and Architecture Governance (governing change toward the target) (pp. 568-569).
- **Key difference.** Within-cycle iteration deepens work inside phases "without affecting the overall flow" of the ADM (p. 569).

Jager's commentary (author's view): His example has two goals (new regulation, efficient HR use) and two initiatives with equal cost: hire staff (Inflow Support capability) or buy compensation software. Both scenarios are developed in B and C and presented to the CIO in Phase F, who picks one via Architecture Alternatives and Trade-Offs (Figure 13-4). A further Architecture Development iteration then refines the chosen scenario (pp. 569-571).

13.3.2 How to Apply Iteration (pp. 573-580)

Iteration group	Purpose	ADM phases	Page
Architecture Capability	Establish and grow the capability: approach, principles, scope, vision, governance	Preliminary, A	p. 573
Architecture Development	Develop architecture content: baseline via B-D, then feasibility via E-F	B to F	p. 573
Transition Planning	Plan the transition and roadmaps from baseline to target	E, F	p. 574
Architecture Governance	Establish and monitor governance	G, H	p. 574

- **Preliminary and A are not repeated routinely.** Approach, principles, vision, and scope don't need redoing for every piece of work (p. 576, Table 13-1).
- **Architecture Development cycles (Figure 13-8):**
  - *Baseline first.* Iteration 1 defines the baseline across B-D (with E/F to test feasibility). Iteration 2 defines the target and the gaps. Further iterations refine (pp. 576-577).

  - *Target first.* Map the desired state first, then compare with the baseline. "A common starting point in many organizations" (p. 577).
- **Transition Planning cycles (Figure 13-9).** Iteration 1 agrees on improvement opportunities against a provisional Transition Architecture, gains buy-in to a solution portfolio, and produces a provisional migration plan. Iteration 2 agrees on the Transition Architecture and refines the plan, feeding issues back to Phase E (pp. 577-578).
- **Architecture Governance cycles (Figure 13-10).** Iteration 1 mobilizes governance and change management processes, people, and technology. Later iterations run periodic compliance reviews, and change results may feed back to the Preliminary Phase or the development phases (pp. 578-579).
- **Treat the steps as a checklist.** Forcing a single linear pass produces "convoluted, looping phase diagrams" (p. 579).

13.4 Architecture Partitioning (pp. 580-583)

- **Definition.** Dividing an EA into smaller, manageable parts to improve governance, scalability, and efficiency, because large organizations can't be architected as one monolith (p. 580).

13.4.1 Why Use Architecture Partitioning (pp. 580-582)

- **Different governance per area.** Domains have different stakeholders, policies, and compliance needs (p. 580).
- **Parallel teams, different timeframes.** Avoids bottlenecks. Architecture can evolve incrementally, with the ADM applied iteratively per partition (p. 581).
- **Definitions per team** prevent architecture that doesn't integrate. Partitioning resembles agile in breaking work into units (p. 581).
- **Keep the whole in view.** Without the broader context you get disjointed components (Figure 13-11) (pp. 582-583).

13.4.2 Supporting Architecture Partitioning (p. 583)

Decided in the Preliminary Phase	Content
Organizational structure	Which teams create which parts of the architecture
Responsibilities for the teams	An agreed definition: subject matter areas, level of detail, delivery time periods
Relationship between architectures	Work toward one integrated architecture. Resolve overlaps so reusable building blocks remain

(p. 583)

13.5 Summary (pp. 583-587)

- Three scenarios: architecture styles, iteration, partitioning (p. 584).
- Three ways to iterate and four iteration groups. Within-cycle iteration loops between phases to refine the architecture over its lifecycle (pp. 585-586).
- Partitioning enables tailored governance, parallel work, and incremental evolution, but requires a holistic view (pp. 586-587).

Key Terms & Definitions

Term	Definition	Page
Architecture style	A distinctive approach (e.g., SOA, microservices, zero-trust) with its own focus, form, and techniques	p. 563
Iteration across the whole cycle	Repeating the full ADM for different levels, scopes, or layers	p. 566
Iteration within a cycle	Looping between two or more phases	p. 567
Architecture Capability iteration	Re-running the Preliminary Phase and A to adjust the capability	pp. 571-572
Iteration groups	Architecture Capability, Architecture Development, Transition Planning, Architecture Governance	pp. 573-574
Baseline first / Target first	Starting Architecture Development iterations with the current or the desired state	pp. 576-577
Architecture partitioning	Dividing the EA into manageable parts, often across teams and timeframes	p. 580

Figures & Tables Referenced

Figure / Table	What it shows	Page
Figure 13-1	The ADM graphic	p. 565
Figure 13-2	Three most common iteration groups within a cycle	p. 568
Figures 13-3, 13-4	Goals and initiatives. Two scenarios via Alternatives and Trade-Offs	pp. 569, 571
Figure 13-5	Architecture Capability iteration group	p. 572
Figure 13-6	Iteration groups of the ADM	p. 574
Table 13-1	Activity by iteration cycle	p. 575
Figures 13-7 to 13-10	Iteration cycles for Architecture Development, Transition Planning, Governance	pp. 576-578
Figure 13-11	Architecture created without an overall picture	p. 582

Self-Check Questions

Q: Why is the ADM graphic often misunderstood?

A: People read it as a linear, waterfall process. It actually shows information flows between logically grouped activities, not a sequence (p. 565).

Q: Name the three ways to apply iteration to the ADM.

A: Iteration across the whole ADM cycle, iteration within an ADM cycle, and iteration to manage the Architecture Capability (p. 566).

Q: Name the four iteration groups and their phases.

A: Architecture Capability (Preliminary, A); Architecture Development (B-F); Transition Planning (E, F); Architecture Governance (G, H) (pp. 573-574).

Q: Why are the Preliminary Phase and Phase A not part of the common iterations?

A: Establishing the approach, principles, vision, and scope does not need repeating for every piece of architecture work (p. 576).

Q: Contrast "baseline first" and "target first".

A: Baseline first defines the current state in iteration 1, then the target and gaps in iteration 2. Target first maps the desired state first, then compares it with the baseline. Target first is common in many organizations (pp. 576-577).

Q: How should TOGAF be adapted to a specific architecture style?

A: Identify the style's unique characteristics, then address them mainly by adapting models, views, and tools (possibly extending the metamodel and Content Framework), not by changing the framework (p. 563).

Q: What triggers an Architecture Capability iteration?

A: Projects wanting to (re)establish aspects of the capability, or new or changed capability requirements, for example from a Phase H change request (p. 572).

Q: Why partition an architecture?

A: Different areas have different stakeholders, policies, and compliance requirements. Partitioning allows targeted governance, parallel teams, and incremental evolution (pp. 580-581).

Q: What three things must be settled in the Preliminary Phase to support partitioning?

A: The organizational structure (teams), each team's responsibilities (subject matter, detail, time periods), and the relationships between architectures (integration and reuse) (p. 583).

Q: What risk does partitioning carry, and how is it avoided?

A: Disjointed components that don't integrate. Avoid it by keeping a holistic view and agreeing definitions and relationships up front (pp. 581-583).